Домашнее задание № 5
«Управление ресурсами кластера: Kubernetes»

Расчёт пар навыков в кластере Kubernetes

Job и Indexed Job, предел памяти по измеренному пику, RustFS внутри кластера Pending, вытеснение по приоритету и квота пространства имён

Выполнил: **Тищенко Павел Валерьевич**
Группа ИИИ-3-ИнжДан-1-1 (J3310), № 16
Репозиторий: `https://github.com/paulhowever/vk_data_eng_2026_ai_67_tischenko`

2026

1 Стенд

Таблица 1. Стенд (hw5/logs/00_cluster.txt, hw5/logs/00_image.txt)

Машина	Apple M5, 10 ядер, 16 ГБ RAM, macOS; Docker Desktop, Engine 29.2.0, виртуальной машине выделено 10 CPU и 7,65 ГиБ
Кластер	kind v0.33.0, Kubernetes v1.37.0, containerd 2.3.4; узлы hh-lake-control-plane, hh-lake-worker, hh-lake-worker2 (hw5/k8s/kind.yaml). У каждого узла allocatable 10 CPU и 8 024 724 Ki памяти, на control-plane taint node-role.kubernetes.io/control-plane
Образ	hhlake-pairs:hw5 (hw5/Dockerfile): Python 3.12, зависимости из uv.lock, пакет hhlake из ДЗ 3 и программа hw5/app/k8spairs.py; пользователь 10001
Хранилище	RustFS 1.0.0 (rustfs/rustfs:1.0.0) в StatefulSet rustfs, Service rustfs.hh-lake.svc.cluster.local:9000, бакет hh-lake
Данные	raw/it_vacancies_2024.csv, 847 619 384 Б; эталон — results/mapreduce/k2.csv из ДЗ 3

Все журналы лежат в hw5/logs/, номер в имени файла — порядок запуска. Почти каждый журнал Job собран скриптом hw5/scripts/capture.sh: get job, describe job, get pods -o wide, describe pod, состояние контейнера, логи пода и события. Смену состояний подов во времени записывает hw5/scripts/watch.sh (kubectl get pods -w с отметкой времени UTC).

2 Программа и образ

Код расчёта я не переписывал. Программа k8spairs.py берёт из ДЗ 3 выравнивание диапазона байт по началу записи (hhlake.mr.split.read_split), фильтр варианта, нормализацию навыков и перебор пар (hhlake.mr.jobs.map_rows), сериализацию и слияние частей (jobs.shuffle_blobs, jobs.merge_blob), вывод CSV (jobs.render_csv) и контрольные числа jobs.EXPECTED, jobs.CONTROL. Режимов три:

- full — весь объект в одном процессе, чанками по 32 МиБ, как в ДЗ 3; результат в results/k8s/full/k2.csv;
- map — часть i из N (байты $[\lfloor iS/N \rfloor, \lfloor (i+1)S/N \rfloor)$, S — размер объекта), номер берётся из JOB_COMPLETION_INDEX; часть пишет tmp/k8s/<run>/part-NNN.jsonl и meta-NNN.json;
- reduce — читает все части, проверяет, что выровненные диапазоны без зазоров и пересечений покрывают файл от конца заголовка до конца объекта, сводит счётчики и пишет results/k8s/<run>/k2.csv.

В конце каждый режим печатает memory.peak и memory.max из cgroup v2 контейнера (/sys/fs/cgroup) и ru_maxrss процесса. Тест tests/test_k8spairs.py проверяет на синтетическом CSV с переводами строк и запятыми внутри полей, что 1, 3 и 8 частей при чанках 512 Б, 4 КиБ и 1 МиБ дают те же пары, что и прямой подсчёт.

Образ собирается из корня репозитория (docker build -f hw5/Dockerfile), контекст ограничен файлом hw5/Dockerfile.dockerignore. Пакеты, которые программе не нужны (pandas, matplotlib, duckdb, kazoo, psutil), не ставятся. В kind образ загружен через docker save --platform linux/arm64 и kind load image-archive: kind load docker-image падает с content digest ... not found, потому что Docker Desktop с containerd image store хранит многоплатформенный индекс без части платформ.

3 RustFS внутри кластера

Манифест hw5/k8s/rustfs.yaml повторяет ДЗ 2 средствами Kubernetes: StatefulSet из одного пода и четыре volumeClaimTemplates disk1..disk4 по 2 Gi в StorageClass standard (local-path). Переменные окружения те же, что в docker-compose.yml: RUSTFS_VOLUMES=/data/disk{1...4}, RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true. Ключи передаются только через Secret hh-lake-s3. Его создаёт hw5/scripts/secret.sh: две строки RUSTFS_*_KEY из .env во временный файл, затем kubectl create secret generic --from-env-file. В репозитории лежит только

`hw5/k8s/secret.example.yaml` с заглушками. Под RustFS просит 500m CPU и 512Mi, предел памяти 2Gi.

Датасет загружен через `kubectl port-forward svc/rustfs 29000:9000` штатным модулем ДЗ 3 с переопределённым адресом (`hw5/logs/01_upload.txt`):

```
$ S3_ENDPOINT=http://localhost:29000 uv run python -m hhlake.upload
local size   847,619,384 B
remote size  847,619,384 B  etag "0e6d21890db664d9bf83e1427665b106-13"
upload time  6.66 s  (121 MiB/s)
sha256 local  303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
sha256 remote 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
integrity     OK
```

Раскладку проверяет `hw5/scripts/storage.sh` (`hw5/logs/02_rustfs_storage.txt`): на каждом из четырёх дисков лежит 423 835 580 Б файлов-частей объекта, то есть 0,5000 объекта, а всего 2,0001× его размера. Это EC 2+2, как в ДЗ 2: два блока данных и два блока чётности.

4 Job с requests/limits и предел памяти

4.1 OOMKilled

Первый запуск `hw5/k8s/job-full-oom.yaml`: режим `full`, `requests`/`limits` по памяти 128Mi, CPU 500m/1, `backoffLimit: 0`. Предел я взял «на глаз»: программа читает файл чанками по 32 МиБ и ничего крупного не копит. Под прошёл 16 чанков из 26 и был убит ядром (`hw5/logs/03_job_full_oom.txt`):

```
  chunk [469,763,676, 503,319,387)  records 9,071  variant 203
  chunk [503,319,387, 536,875,223)  records 9,474  variant 297
$ kubectl -n hh-lake describe pod/pairs-full-oom-h4j97
    State:          Terminated
      Reason:       OOMKilled
      Exit Code:    137
      Started:      Tue, 10 Nov 2026 12:36:25 +0300
      Finished:     Tue, 10 Nov 2026 12:36:28 +0300
    Limits:
      cpu:     1
      memory:  128Mi
  Warning  BackoffLimitExceeded  1s    job-controller  Job has reached the specified backoff limit
```

Код 137 = 128 + 9: процесс получил SIGKILL от OOM killer, потому что память cgroup контейнера дошла до `memory.max` = 128 МиБ.

4.2 Измерение пика

Тот же Job с пределом 1Gi (`hw5/k8s/job-full-measure.yaml`) я запустил три раза подряд (`hw5/logs/04_job_full_measure_1..3.txt`). Все три прогона прошли все контрольные проверки ДЗ 3.

Таблица 2. Пик памяти полного расчёта, предел 1Gi

Прогон	`memory.peak` cgroup, МиБ	`ru_maxrss`, МиБ	время расчёта, с
1	**163,9**	145,3	5,1
2	138,3	145,3	5,5
3	150,9	145,1	4,5

Пик я беру по `memory.peak` cgroup, а не по `ru_maxrss`: OOM killer сравнивает с `memory.max` именно память cgroup. В неё кроме анонимной памяти процесса входят страничный кэш прочитанных файлов образа (модули Python, библиотеки) и память ядра под сокеты. Поэтому `memory.peak` колеблется от прогона к прогону сильнее, чем `ru_maxrss`: скорее всего, в первом прогоне кэш файлов образа был впервые учтён в этой cgroup, а в следующих часть страниц уже числилась за cgroup прошлых подов. Разброс 138–164 МиБ, то есть ±9% вокруг среднего 151 МиБ.

Расчёт предела. $P = \max(163{,}9;\ 138{,}3;\ 150{,}9) = 163{,}9$ МиБ. Запас $k = 1{,}25$:

$$L = \left\lceil \frac{P \cdot k}{16} \right\rceil \cdot 16 = \left\lceil \frac{163{,}9 \cdot 1{,}25}{16} \right\rceil \cdot 16 = \lceil 12{,}8 \rceil \cdot 16 = 208 \text{ МиБ}.$$

Почему 25%: (1) разброс самого пика между прогонами около 9%, а три прогона — маленькая выборка; (2) пик зависит от данных: самый длинный чанк, число уникальных пар в счётчике, фрагментация кучи Python; в пределе 128Mi процесс упал на 17-м чанке, то есть память росла по ходу расчёта; (3) повторные запросы boto3 и буферы сокетов при медленном S3. Больше 25% не беру: память здесь — то, по чему считается квота и упаковка подов на узел (раздел 6.3: 512Mi квоты вмещают ровно два пода по 208Mi). `requests` = `limits` = 208Mi, чтобы планировщик резервировал ровно то, что под может занять.

С пределом 208Mi Job `pairs-full` прошёл: контейнер завершился с `reason=Completed` и `exitCode=0`, `memory.peak` составил 145,3 МиБ, то есть 70% предела, а `memory.max` в контейнере был 208,0 МиБ (журнал `hw5/logs/05_job_full_limit.txt`). Части Indexed Job с тем же пределом заняли 105,8–131,2 МиБ (`hw5/logs/06_indexed_map.txt`), сведение — 54,3 МиБ.

5 Indexed Job по частям файла

`hw5/k8s/job-indexed.yaml`: `completionMode: Indexed`, `completions: 8`, `parallelism: 4`, `backoffLimitPerIndex: 2`, ресурсы как у `pairs-full`. После него `hw5/k8s/job-reduce.yaml` сводит части. Порядок «map, затем reduce» задаю я сам через `kubectl wait --for=condition=Complete`: у Job нет зависимостей между заданиями.

```
$ kubectl -n hh-lake logs job/pairs-reduce                                    (hw5/logs/07_reduce.txt)
part  0 node hh-lake-worker   aligned [222, 105,958,703) records 28,564 variant  502 keys 3,549 time 1.6 s peak 107.7 MiB
part  1 node hh-lake-worker2  aligned [105,958,703, 211,909,622) records 29,196 variant  556 keys 4,398 time 1.6 s peak 131.2 MiB
part  2 node hh-lake-worker   aligned [211,909,622, 317,857,720) records 28,739 variant  476 keys 3,852 time 1.5 s peak 107.5 MiB
part  3 node hh-lake-worker   aligned [317,857,720, 423,810,887) records 29,086 variant  733 keys 5,212 time 1.6 s peak 107.9 MiB
part  4 node hh-lake-worker   aligned [423,810,887, 529,763,672) records 28,915 variant  727 keys 4,697 time 1.1 s peak 106.3 MiB
part  5 node hh-lake-worker2  aligned [529,763,672, 635,715,091) records 28,444 variant  823 keys 6,288 time 1.1 s peak 105.8 MiB
part  6 node hh-lake-worker   aligned [635,715,091, 741,667,049) records 28,400 variant  734 keys 5,199 time 1.2 s peak 106.4 MiB
part  7 node hh-lake-worker   aligned [741,667,049, 847,619,384) records 28,044 variant  831 keys 6,016 time 1.1 s peak 106.3 MiB
check OK   aligned ranges tile [header_end, size): True (expected True)
check OK   records: 229388 (expected 229388)
check OK   variant: 5382 (expected 5382)
check OK   with skills: 4380 (expected 4380)
check OK   k=2 sum(count) == emitted: 135362 (expected 135362)
check OK   k=2 unique keys: 15921 (expected 15921)
check OK   k=2 sum count: 135362 (expected 135362)
check OK   k=2 top: (('mysql', 'php'), 2096) (expected (('mysql', 'php'), 2096))
written s3://hh-lake/results/k8s/indexed/k2.csv  467,629 B
```

Первая часть начинается с байта 222 — это конец заголовка, последняя кончается на 847 619 384. Соседние выровненные диапазоны стыкуются, поэтому каждая запись прочитана ровно одним подом.

Время. Job `pairs-map` шёл 10 с: `startTime` 09:37:48, `completionTime` 09:37:58 (`hw5/logs/12_jobs_summary.txt`). Поды шли двумя волнами по четыре: 09:37:48–09:37:50 и 09:37:53–09:37:55. Сам расчёт части занимает 1,1–1,6 с, остальное — создание подов и запуск контейнеров. Полный расчёт в одном поде (`pairs-full`) занял 4,7 с счёта и 8 с от старта до завершения Job. На файле в 850 МБ параллельность в Kubernetes не ускоряет расчёт: накладные расходы на под (около 3 с) больше, чем выигрыш от деления 5 с счёта на четыре.

Сверка. `hw5/scripts/verify_k2.py` скачивает результаты через port-forward и сравнивает с эталоном ДЗ 3 в DuckDB (`hw5/logs/08_verify_k2.txt`):

```
s3://hh-lake/results/k8s/indexed/k2.csv -> hw5/results/indexed/k2.csv
  reference results/mapreduce/k2.csv: rows 15,921 sum 135,362
  kubernetes: rows 15,921 sum 135,362
  k8s EXCEPT ALL ref: 0 rows
  ref EXCEPT ALL k8s: 0 rows
  top-10: mysql+php 2096, git+php 1686, laravel+php 1342, git+mysql 1116, javascript+php 1103, php+postgresql 975,
     docker+php 907, php+symfony 872, html+php 839, php+sql 800
ab85544b061cf897d33311e8e7a82b584e68e33bf9e8701673ca846ae6d19e3f  results/mapreduce/k2.csv
ab85544b061cf897d33311e8e7a82b584e68e33bf9e8701673ca846ae6d19e3f  hw5/results/indexed/k2.csv
ab85544b061cf897d33311e8e7a82b584e68e33bf9e8701673ca846ae6d19e3f  hw5/results/full/k2.csv
```

Результаты Indexed Job и полного Job совпадают с ДЗ 3 и как множества строк (0/0 в обе стороны),

и побайтно: sha256 одинаковый.

6 Планирование

6.1 Pending при нехватке CPU

`hw5/k8s/job-pending-cpu.yaml` — тот же Indexed Job, но каждый под просит 6 CPU при 10 CPU на узле, `parallelism: 3`. Рабочих узлов два, на каждый помещается один такой под, а control-plane закрыт taint (`hw5/logs/09_pending_cpu.txt`):

```
$ kubectl -n hh-lake get pods -l batch.kubernetes.io/job-name=pairs-map-cpu -o wide   (3 s after apply)
NAME                   READY   STATUS    RESTARTS   AGE   IP            NODE              NOMINATED NODE
pairs-map-cpu-0-mw5s4  1/1     Running   0          2s    10.244.2.14   hh-lake-worker    <none>
pairs-map-cpu-1-9hqpv  0/1     Pending   0          2s    <none>        <none>            <none>
pairs-map-cpu-2-tp8jz  1/1     Running   0          2s    10.244.1.9    hh-lake-worker2   <none>
  Warning  FailedScheduling  2s    default-scheduler  0/3 nodes are available: 1 node(s) had untolerated taint(s), 2
     Insufficient cpu. preemption: 0/3 nodes are available: 1 Preemption is not helpful for scheduling, 2 No
     preemption victims found for incoming pod.
```

Под 1 ждал 4 с и встал на `hh-lake-worker`, как только там закончился под 0 (`watch`: 09:38:49 создан, 09:38:53 назначен). Дальше Job так и шёл по два пода: 8 частей заняли 18 с против 10 с у `pairs-map`, где под просил 500m. Планировщик смотрит только на `requests`: реальной нагрузки на CPU здесь почти нет, программа однопоточная.

6.2 Вытеснение по приоритету

`hw5/k8s/priority.yaml` создаёт два PriorityClass: `batch-low` (1000, `preemptionPolicy: Never`) и `batch-high` (100 000, `PreemptLowerPriority`). Job `pairs-low` (два пода по 6 CPU, каждый 20 раз подряд считает полный файл) занимает оба worker. Через 9 с запускается `pairs-high`: один под на 6 CPU, один полный расчёт (`hw5/logs/10_preemption.txt`):

```
$ kubectl -n hh-lake get pods -l run in (low,high) -o wide   (3 s after high)
NAME              READY   STATUS        RESTARTS   AGE   IP            NODE              NOMINATED NODE
pairs-high-86vtd  0/1     Pending       0          3s    <none>        <none>            hh-lake-worker
pairs-low-97bw2   1/1     Terminating   0          12s   10.244.2.18   hh-lake-worker    <none>
pairs-low-z4t67   1/1     Running       0          12s   10.244.1.13   hh-lake-worker2   <none>
2026-11-10T09:40:39Z   Warning   FailedScheduling   pairs-high-86vtd   0/3 nodes are available: 1 node(s) had
     untolerated taint(s), 2 Insufficient cpu. preemption: found a potential placement for pod on node hh-lake-worker
     , preempting 1 victims
2026-11-10T09:40:39Z   Normal    Killing            pairs-low-97bw2    Stopping container pairs
2026-11-10T09:40:39Z   Normal    Preempted          pairs-low-97bw2    Preempted by pod ea3b662e-a481-4c6f-88a4-9
     cdd02d18f65 on node hh-lake-worker
$ kubectl -n hh-lake get pod pairs-high-86vtd -o jsonpath={.metadata.uid} {.spec.priorityClassName} {.spec.priority}
ea3b662e-a481-4c6f-88a4-9cdd02d18f65 batch-high 100000
```

Хронология по `watch`: в 09:40:39 планировщик выбрал жертву и записал высокоприоритетному поду `nominatedNodeName`, жертва получила SIGTERM. В 09:40:44, через `terminationGracePeriodSeconds: 5`, она завершилась (`Error`), и в 09:40:45 `pairs-high` запустился на освободившемся узле. В 09:40:50 он закончил расчёт. Job `pairs-low` создал замену вытесненному поду в 09:40:54, она начала свои 20 проходов с нуля. Итог `pairs-low`: `succeeded: 2`, без `failed`, 117 с вместо примерно 95 с у невытесненного пода. Вытеснение не засчитано как сбой, потому что в Job стоит `podFailurePolicy` с правилом `Ignore` для условия `DisruptionTarget`. Без него вытеснение тратило бы `backoffLimit`.

6.3 ResourceQuota пространства имён

`hw5/k8s/quota.yaml` создаёт пространство `team-php` с квотой `pods: 4`, `requests.cpu: 2`, `requests.memory: 512Mi`, `limits.cpu: 4`, `limits.memory: 512Mi` и LimitRange: по умолчанию контейнер получает request 500m/208Mi и limit 1/208Mi, больше 512Mi одному контейнеру нельзя. В `job-quota.yaml` ресурсов нет вовсе, их подставляет LimitRange. `parallelism: 4`, но по памяти в квоту помещаются только два пода: $2 \cdot 208 = 416 \le 512$, а $3 \cdot 208 = 624 > 512$ (`hw5/logs/11_quota.txt`):

```
$ kubectl -n team-php describe resourcequota team-php-quota          (2 s after apply)
limits.memory    416Mi  512Mi
pods             2      4
```

```
requests.memory  416Mi  512Mi
  Warning  FailedCreate  59s  job-controller  Error creating: pods "pairs-map-quota-2-zfmtx" is forbidden: exceeded
     quota: team-php-quota, requested: limits.memory=208Mi,requests.memory=208Mi, used: limits.memory=416Mi,requests.
     memory=416Mi, limited: limits.memory=512Mi,requests.memory=512Mi
$ kubectl -n team-php run quota-probe ... "requests":{"memory":"400Mi"},"limits":{"memory":"400Mi"}
Error from server (Forbidden): pods "quota-probe" is forbidden: exceeded quota: team-php-quota, requested: limits.
     memory=400Mi,requests.memory=400Mi, used: limits.memory=416Mi,requests.memory=416Mi, limited: limits.memory=512
     Mi,requests.memory=512Mi
```

Квота срабатывает при создании пода, а не при планировании: API-сервер отклоняет запрос, под даже не появляется в Pending. Job-контроллер повторяет создание с задержкой. В журнале 10 отклонённых попыток, и одновременно в `team-php` работало не больше двух подов. 8 частей заняли 18 с, как при нехватке CPU, все 8 завершились успешно.

7 Ответы на вопросы

1. Что в Kubernetes соответствует ResourceManager, NodeManager, ApplicationMaster и Container из YARN. ResourceManager — это kube-apiserver с etcd (состояние кластера и учёт ресурсов) и kube-scheduler (выбор узла). NodeManager — kubelet с containerd на каждом узле: он запускает контейнеры, ставит им cgroup-пределы и сообщает их состояние. Container — под (точнее, контейнер пода), его `requests`/`limits` — это запрос контейнера в YARN (память и vcores). ApplicationMaster ближе всего к контроллеру Job, но с важным отличием. AM в YARN свой у каждого приложения, работает в контейнере и сам ведёт приложение: просит контейнеры, перезапускает задачи, переводит map в reduce. Контроллер Job один на кластер и знает только «запусти N подов до успеха». В моём запуске он создал `pairs-map-0..7`, четыре сразу и ещё четыре по мере завершения, и повторил бы упавший индекс (`backoffLimitPerIndex: 2`). Но переход к reduce сделал я, командой `kubectl wait`. Роль AM с логикой приложения в Kubernetes играют операторы (Spark operator, Argo Workflows) или драйвер Spark в собственном поде.

2. Двухуровневое планирование Mesos против kube-scheduler. В Mesos мастер сам не размещает задачи. Его аллокатор делит ресурсы между фреймворками по DRF (выравнивает доминантные доли: у кого больше доля по самому дефицитному ресурсу, тот получает меньше) и отправляет фреймворкам предложения (offers) с ресурсами конкретных агентов. Планировщик фреймворка решает, принять предложение и что на нём запустить, или отказаться. Так каждый фреймворк (Spark, Marathon) планирует по своей логике, но видит только то, что ему предложили. Если задаче нужно 6 CPU на одном агенте, фреймворк отклоняет мелкие предложения и ждёт. kube-scheduler один и видит весь кластер: для каждого пода он отфильтровывает узлы (ресурсы, taints, affinity), оценивает оставшиеся и связывает под с лучшим. Поэтому сообщение `0/3 nodes are available: 1 node(s) had untolerated taint(s), 2 Insufficient cpu` перечисляет все три узла с причинами. Справедливого деления между арендаторами вроде DRF в kube-scheduler нет. Есть приоритеты, вытеснение и квоты, а очереди с долями добавляют надстройки (Kueue, Volcano, Apache YuniKorn).

3. Очереди Capacity/Fair Scheduler против Namespace + ResourceQuota/LimitRange. Очередь YARN даёт гарантию и делится ресурсами. У очереди есть гарантированная доля кластера (capacity), простаивающие ресурсы других очередей можно занять до maximum-capacity, Fair Scheduler выравнивает доли со временем. Приложение, которому не хватает места, ждёт в очереди (ACCEPTED), и планировщик запустит его сам. ResourceQuota — жёсткий потолок суммы `requests`/`limits` в пространстве имён. Она проверяется при создании объекта, ничего не гарантирует (квоту можно выставить больше, чем есть в кластере), занимать чужое не даёт и очереди не держит. В моём запуске лишние поды `team-php` не ждали в очереди, а получали `exceeded quota`, и ожидание обеспечивал Job-контроллер, повторяя попытки. LimitRange соответствует `yarn.scheduler.minimum/maximum-allocation-mb` и размеру контейнера по умолчанию: `job-quota.yaml` вообще без ресурсов получил 208Mi из LimitRange. Ближе всего к очередям YARN в Kubernetes Kueue: ClusterQueue с квотой, одалживание внутри cohort, приём заданий в порядке очереди.

4. Вытеснение в YARN против PriorityClass. В Capacity Scheduler вытеснение восстанавливает гарантии очередей. Если очередь ниже своей capacity, а другая заняла её ресурсы, политика

вытеснения сначала шлёт AM чужой очереди предупреждение, а после тайм-аута убивает контейнеры. Fair Scheduler так же возвращает очередям их справедливую долю. Вытесненный контейнер завершается со статусом PREEMPTED и не считается сбоем приложения. В Kubernetes вытеснение идёт по приоритету пода, а не по долям: когда под с большим приоритетом не помещается, планировщик ищет узел, где удаление подов с меньшим приоритетом освободит место, и вытесняет их с graceful termination. В моём запуске `pairs-high` (100 000) вытеснил `pairs-low-97bw2` (1000) за 5 с `terminationGracePeriodSeconds`. Как и в YARN, вытеснение не стало сбоем, но только благодаря `podFailurePolicy` с правилом `Ignore` для `DisruptionTarget`. Как и в YARN без чекпоинтов, работа вытесненного пода пропала: замена начала с первого прохода. Сами поды `batch-low` никого вытеснить не могут, у класса `preemptionPolicy: Never`: фоновые задания ждут, но никого не выталкивают.

5. Изоляция через cgroups. И YARN, и Kubernetes, и Mesos ограничивают задачи средствами ядра Linux. В YARN это LinuxContainerExecutor с CGroupsHandler (cpu, memory) и, кроме того, монитор NodeManager: он периодически суммирует память дерева процессов контейнера и убивает контейнер, превысивший предел (*running beyond physical memory limits*, код 143). У Mesos этим занимаются изоляторы `cgroups/cpu` и `cgroups/mem` контейнеризатора. В Kubernetes kubelet через containerd/runc создаёт cgroup v2 на под и на контейнер: `memory.max` равен limit, `cpu.max` задаётся по limit CPU, `cpu.weight` — по request. Превышение памяти ловит само ядро, а не опрос. У меня это видно напрямую: `memory.max` внутри контейнера был 128, 1024 и 208 МиБ в трёх Job, при 128 МиБ ядро убило процесс (OOMKilled, 137), а `memory.peak` той же cgroup дал число для расчёта предела. CPU-limit ядро только притормаживает (throttling) и процесс не убивает, а память — несжимаемый ресурс, поэтому её предел и пришлось измерять.

6. Где какая система уместна. YARN — кластер Hadoop, где данные лежат в HDFS на тех же машинах (планировщик учитывает локальность), а нагрузка — Spark, Hive и MapReduce многих команд с гарантированными долями очередей. Mesos — общий пул машин для нескольких фреймворков со своими планировщиками. Сейчас эту нишу почти целиком занял Kubernetes, а сообщество Mesos практически перестало развивать проект. Kubernetes — контейнерные сервисы и пакетные задания на одном кластере, с декларативным API, хранилищем через PVC (как RustFS в StatefulSet здесь) и одинаковым запуском на ноутбуке (kind) и в облаке. Для пакетной нагрузки с очередями и справедливым делением к нему добавляют Kueue, Volcano или YuniKorn. На моих данных (850 МБ, расчёт около 5 с в одном поде) Kubernetes не ускорил расчёт: накладные расходы на под больше выигрыша от параллельности. Зато он дал воспроизводимый образ, явные пределы памяти с проверкой ядром, повтор упавших частей, приоритеты и квоты.

8 Замечания вне задачи

- `kubectl` 1.34.1 при сервере 1.37.0: в `hw5/logs/00_cluster.txt` предупреждение о расхождении версий больше чем на одну минорную. Все команды работы прошли, но поведение не гарантировано. Вред P3, исправление — обновить `kubectl`.
- Образ RustFS в ДЗ 2 указан как `rustfs/rustfs:latest`; здесь он закреплён как `rustfs/rustfs:1.0.0` (тот же образ, `sha256:8cc98017…`). Вред P2 для ДЗ 2: при новом `latest` стенд ДЗ 2 может подняться с другой версией.

9 Итог

Кластер kind из трёх узлов, образ с программой ДЗ 3 и RustFS с EC 2+2 в StatefulSet работают. Job с пределом памяти 128Mi получил OOMKilled (exit 137). По трём замерам `memory.peak` (максимум 163,9 МиБ) и запасу 25% выведен предел 208Mi, с ним полный Job проходит при пике 145,3 МиБ. Indexed Job из 8 частей по 4 пода за 10 с и сведение дали k2, совпадающий с ДЗ 3: топ-10 тот же, `EXCEPT ALL` пуст в обе стороны, sha256 одинаковый. Показаны Pending по CPU (`FailedScheduling`, `Insufficient cpu`), вытеснение `pairs-low` подом `pairs-high` (`Preempted`) и отказ по квоте (`exceeded quota`).